

Implementing JWT Web API security in .NET 9

Source code [here](#)

Front-end related article [here](#)

In this article we are going to explain how to provide authentication and authorisation capabilities in .NET 9 Core based Web API that supplies access to various functionalities of a real-life application, protecting it by implementing JWT authentication and storing user credentials in a MongoDB database.

This is a straight-forward procedure easy to implement and offers a nice layer of security that complies with industry standards for web development.

Author of the article: *Iker Elg. Alzaa*

#Cybersecurity #NET #JWT #WebAPI
#Identity #Mongo



Authentication is the process of validating user credentials and authorisation is the process of checking privileges for a user to access specific modules in an application.

For this purpose, we protect a .NET Web API application by implementing JWT authentication in both sides, the front and back-end. This article covers the back-end side, being available the front-end the next articles that will be released.

JSON Web Token (JWT)

JSON Web Token (JWT) is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object.

JSON Web Tokens consist of three parts separated by dots, which are:

- Header
- Payload
- Signature

Therefore, a JWT looks like the following:

```

Body 200 OK 948 ms 1.75 KB
{ } JSON
1 {
2   "Token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzb2FwLn9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2NsYWltcy9uYXN1IjoiaWRTaW4iLCJqdGkiOiIzMjRiODc3Mi03MmI4LTQyNjgtOTFkZC0yNTdiNDM2NTBhZGUilCJodHRwOi8vc2NoZW1hcy5taWYybnV3ZnZuY29tL3dzLzIwMDgvMDYvaWRlbnRpdHkvY2xhaW1zL3JvbGUiOiJBZG1pb2IiLCJpbiIsImV4cCI6MTc2NzUyNTEuNCwiaXNzIjoiaHR0cDovL2xvY2FsaG9zdDo0MDAwIiwiaWF0IjoiYXN1IjoiaHR0cDovL2xvY2FsaG9zdDo0MDAwIn0.DUyXProVHb_I502bsz-noTF4RnzcSAfsCjvgD0U_5JM",
3   "RefreshToken": "9TreV1bEDtr6TowdRE5P2ldwXjhpv5SewHfNGfn3hYcgqr3f1Mx0M5Ct/vDrcYVjwLHYki0WwNTMw8Go9mDVxA=",
4   "Expiration": "2026-01-04T12:34Z",
5   "Email": "admin@example.com",
6   "Roles": [
7     "Admin"
8   ]
9 }

```

This information can be verified and trusted because it is digitally signed using a secret (with the HMAC algorithm) or a public/private key pair using RSA or ECDSA.

To implement the JWT authentication procedure we need to install the next Nuget packages.

- `AspNetCore.Identity.MongoDbCore`, this is a MongoDB UserStore and RoleStore adapter for `Microsoft.Extensions.Identity.Core`
- `Microsoft.AspNetCore.Authentication.JwtBearer`, this package enables the application to receive an OpenID Connect bearer token

Auth. model implementation

After loading the Nugget packages, we start our implementation creating a new folder called *Auth* where to store all the models to manage the authentication and authorisation process. This is the place where we are going to define our Identity "User" and "Role" classes.

The Identity Role class inherits from *MongolidentityRole* class and is mapped to *Roles* collections, the analogous happens with the Identity User. We also create a static class *UserRoles* to add two constant values *Admin* and "User" as roles.

Then create a *User.cs* class for new user registration. In this class we store the personal information for each user (Name, Email and Password). We store the login user credentials (name and password) inside the *LoginModel.cs* class.

To store the information required by the JWT authorisation we create the *TokenModel.cs* class. In this class we store the *AccessToken*

and the *RefreshToken*. The access token is valid for the time period we establish in the *appsettings.json* file. After that period of time, we have to create a new token with the refresh token.

To manage the http requests responses, we create the *Response.cs* class. In this class we define the response message and the response status code.

Configuration (*appsettings.json*)

After creating all the authentication and authorisation models classes, we proceed with the *appsettings.json* file modifications.

We have to include a new section "JWT" inside the file and we define five new variable (*ValidAudience*, *ValidIssuer*, *Secret*, *TokenValidityInMinutes* and *RefreshTokenValidityInDays*) like this:

```
{
  "appsettings.json" > ...
  1  {
  2    "JWT": {
  3      "ValidAudience": "http://localhost:4200",
  4      "ValidIssuer": "http://localhost:5000",
  5      "Secret":
  6        "JWTRefreshTokenHIGHsecuredPasswordVWvp10H7XzYr",
  7      "TokenValidityInMinutes": 120,
  8      "RefreshTokenValidityInDays": 7
  9    },
 10    "BookStoreDatabase": {
 11      "ConnectionString": "mongodb://host.docker.internal:27017/?directConnection=true&serverSelectionTimeoutMS=60000&appName=mongosh+2.5.10",
 12      "DatabaseName": "BookStore",
 13      "BooksCollectionName": "Books"
 14    },
 15    "Logging": {
 16      "LogLevel": {
 17        "Default": "Information",
 18        "Microsoft.AspNetCore": "Warning"
 19      }
 20    },
 21    "AllowedHosts": "*"
 22  }
```

- *ValidAudience*: What is the target of this token. In other words, which services should accept this token as an access token for the service.

- *ValidIssuer*: Who created the token. In this case is our localhost web API. When we launch the API.
- *Secret*: The secret will be used for setting the *JWT IssuerSigningKey*. It is the public key used for validating incoming JWT tokens.
- *TokenValidityInMinutes*: the access token is valid for the period of time established in this variable. After that time the token expires and the user should request a new token with the aid of the refresh token.
- *RefreshTokenValidityInDays*: as an additionally functionality we can set the refresh token expiration time.

We define the *MongoDb*, the *Identity* database, as the database for storing the user's credential (username, roles, etc). The *MongoDb* must be running in three *Docker* containers our localhost in port 27017, 27018 and 27019.

We also configure the *JWT Authentication* system in *Program.cs*.

API controller

After introducing the required modification in the middleware, we are going to create an API controller *AuthenticateController.cs* inside the *Controllers*.

- *RegisterUser*, *RegisterAdmin*: this function receives as a parameter the new user info (username, password and email).
- *Login*: after registering a new user, we have stored the user personal information in the *MongoDB* with the token.
- *Revoke*: in case we would like to revoke the permissions granted to a user, we can set to "null" the *RefreshToken*, then the user cannot login again.

After the Authentication process (*register user, login, create token, etc.*) defined in the *AuthenticateController.cs*, we have to authorise users to access the actions in the *BooksController.cs*.

For this purpose, we include the *[Authorize]* attribute in all the actions where we require a

granted permission to be accessed, together with with the permitted role if needed.